

Deployment Plan for ImaginAI

ImaginAI is a containerized multi-tier web application consisting of:

1. Frontend: Next.js (React 19, Zustand) connecting to a MongoDB Atlas cluster.
2. Backend: FastAPI (Python 3.10) that processes NLP scripts, generates character behaviors, utilizes ChromaDB (vector storage), calls Hugging Face & Groq APIs, and compiles cinematic images into video clips via FFmpeg.

This plan details the configurations, code modifications, and cloud setups required to deploy ImaginAI to a production environment.

User Review Required

IMPORTANT:

- *State & Storage Persistence: The Python backend generates images and videos locally in `/generated_images` and runs ChromaDB using a persistent SQLite client `./chroma_db`.*
- *Recommendation: Deploy the backend as a container with Persistent Disk Volume support (available on Render Web Services, Railway, or VPS). Standard serverless environments (Vercel Serverless, AWS Lambda) are not suitable due to ephemeral storage restarts and size constraints of PyTorch/SentenceTransformers.*
- *Alternative: Set up cloud object storage (AWS S3 or Cloudinary) for images/videos, and a hosted Vector DB (Chroma Cloud, Pinecone, or MongoDB Atlas Vector Search) if you prefer serverless deployments.*
- *Large Dependency Footprint: SentenceTransformers and PyTorch require substantial memory. A CPU-only container builds small enough (~1.5GB) to fit inside basic VPS or Render instances but requires at least 2GB RAM to start up.*

Proposed Changes

Configuration & Infrastructure Files

[NEW] [Dockerfile (Backend)](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/ai-backend/Dockerfile)

Create a Dockerfile to build a lightweight, production-ready backend image with system-wide FFmpeg and pre-downloaded sentence-transformer models (minimizing boot times and eliminating initialization timeouts).

[NEW] [requirements.txt (Backend)](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/ai-backend/requirements.txt)

Define clean dependencies for the FastAPI python application.

[NEW] [Dockerfile (Frontend)](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/Dockerfile)

Create a multi-stage production Docker build for the Next.js web interface.

[NEW] [docker-compose.yml (Root)](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/docker-compose.yml)

Create a docker-compose orchestrator to run both services locally in a containerized network mirroring production conditions.

Code Refactoring to Support Environment Variables

To transition from local development to production, we must replace hardcoded ``http://127.0.0.1:8000`` URLs with configurable environment variables.

[MODIFY] [useStore.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/store/useStore.js)

Update client-side API requests to read from ``process.env.NEXT_PUBLIC_AI_BACKEND_URL``.

[MODIFY] [GalleryView.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/components/GalleryView.js)

Change physical image and video deletion calls from hardcoded URLs to ``process.env.NEXT_PUBLIC_AI_BACKEND_URL``.

[MODIFY] [Next.js API Routes](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api)

Replace hardcoded strings with ``process.env.AI_BACKEND_URL`` in the following server-side routes:

- `[api/videos/[filename]/route.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api/videos/%5Bfilename%5D/route.js)`
- `[api/images/[filename]/route.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api/images/%5Bfilename%5D/route.js)`
- `[api/projects/[projectId]/scenes/route.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api/projects/%5BprojectId%5D/scenes/route.js)`
- `[api/projects/[projectId]/scenes/[sceneId]/generate-images/route.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api/projects/%5BprojectId%5D/scenes/%5BsceneId%5D/generate-images/route.js)`
- `[api/projects/[projectId]/scenes/[sceneId]/generate-clips/route.js](file:///c:/Users/hp/CODEBASE/Projects/ImaginAI/frontend/app/api/projects/%5BprojectId%5D/scenes/%5BsceneId%5D/generate-clips/route.js)`
- `api/characters/upload-avatar/route.js`
- `api/character-summary/route.js`
- `api/character-arc/route.js`

Detailed Step-by-Step Deploy Guide

Phase 1: Environment & Database Setup

1. MongoDB Atlas Cluster:

- Go to MongoDB Atlas and set up a free shared cluster.
- Create a database user (e.g. user `aryan`) and copy the connection string.
- Ensure the IP access whitelist allows incoming requests from your hosting servers (or allow `0.0.0.0/0` for serverless hosts).

2. Third-Party API Keys:

- Obtain a Groq API Key for LLM completions.
- Obtain a Hugging Face Token for FLUX.1-schnell image inference.

Phase 2: Host Deployment Options

Option A: Containerized Cloud Services (Render / Railway) - *Recommended & Easiest*

1. Deploy the Backend (FastAPI)

- Host: Deploy as a Render Web Service or a Railway Service.
- Source: Connect your Git repository. Set the root directory to `ai-backend`.
- Environment Variables:
- `GROQ\_API\_KEY`: (Your Groq Key)
- `HF\_TOKEN`: (Your HF Token)
- Persistent Disk (CRITICAL):
- Add a Persistent Volume/Disk to the service.
- Mount Path: `/app/generated\_images` (size: 5GB-10GB).
- Mount Path (or sub-directory): `/app/chroma\_db` (to store vector memory).
- Scale Requirements: Select an instance with at least 2GB RAM (e.g., Render Starter plan).

2. Deploy the Frontend (Next.js)

- Host: Deploy as a Vercel Project or Render Web Service.
- Root Directory: `frontend`
- Environment Variables:
- `MONGODB\_URI`: `mongodb+srv://...`
- `JWT\_SECRET`: (A random 64-character hex string)
- `AI\_BACKEND\_URL`: `https://your-backend-service-url.onrender.com` (Internal URL can also be used if on the same private network)

- `NEXT\_PUBLIC\_AI\_BACKEND\_URL`: `https://your-backend-service-url.onrender.com` (External URL accessed by the browser)

Option B: Virtual Private Server (VPS) - *Most Cost-Effective for Large Deployments*

Deploying on a VPS (DigitalOcean Droplet, AWS EC2, Linode) running Ubuntu 22.04 LTS.

1. Initial VPS Setup

- SSH into the VPS and install Docker + Docker Compose:

```
``bash
sudo apt update && sudo apt upgrade -y
sudo apt install docker.io docker-compose -y
sudo systemctl enable --now docker
...`
```

2. Deploying with Docker Compose

- Clone the repository on the VPS.
- Create a production `.env` file in the project root:

```
``env
MONGODB_URI=mongodb+srv://...
JWT_SECRET=your_jwt_secret_here
GROQ_API_KEY=your_groq_api_key
HF_TOKEN=your_hugging_face_token
...`
```

- Start the services using the docker-compose config:

```
``bash
docker-compose -f docker-compose.yml up -d --build
...`
```

3. Reverse Proxy (Nginx) & SSL

- Install Nginx and Certbot:

```
``bash
sudo apt install nginx certbot python3-certbot-nginx -y
...`
```

- Configure Nginx (`/etc/nginx/sites-available/imaginai`) to map frontend to port `3000` and proxy backend routes:

```
``nginx
server {
server_name yourdomain.com;
```

```
location / {
    proxy_pass http://localhost:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
}
...

- Enable the configuration and configure SSL:
```bash
sudo ln -s /etc/nginx/sites-available/imaginai /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl restart nginx
sudo certbot --nginx -d yourdomain.com
```
```

Verification Plan

Automated / Integration Checks

1. Container Builds: Validate Dockerfiles by running local compose builds:

```
```bash
docker-compose build
```
```

2. API Health Checks: Curl the deployed backend root `/` and expect:

```
```json
{"message": "Manomaya AI Backend Running"}
```
```

Manual Verification Flow

1. Open the deployed Next.js site in a web browser.
2. Sign up a new user, log in, and create a test project.
3. Add 2 test characters with complete emotional/brain properties.
4. Go to Scene Studio, submit a scene prompt, and wait for completion.
 - Verify the narrative text loads.

- Verify emotional metrics update on character profiles and canvas nodes.
- 5. Click Generate Storyboard and check if images render successfully.
- 6. Verify video clips compile and play in the browser console.
- 7. Restart the backend container and confirm that previous images and memories are retained (verifying persistent disk integration).